

InterviewAI — Models & Technology Reference Guide

Document Type: AI/ML Model Specification & Technology Stack Reference
Purpose: Comprehensive guide for teammates to understand all models, libraries, and architecture
Models Covered: Qwen3-8B • Qwen3-4B • Ollama Qwen2.5:7B • Whisper-v3-turbo • BGE Embeddings • FERPlus ONNX • OpenCV Haar Cascades
Author: Sumit Kumar Sahoo & Team | **Version:** 3.0.0 | **Date:** October 2026

Table of Contents

- 1. AI/ML Models at a Glance
- 2. Model 1 — Qwen3-8B (Primary LLM)
- 3. Model 2 — Qwen3-4B (Fallback LLM)
- 4. Model 3 — Ollama Qwen2.5:7B (Local LLM)
- 5. Model 4 — Whisper-large-v3-turbo (Speech-to-Text)
- 6. Model 5 — BAAI/bge-small-en-v1.5 (Semantic Embeddings)
- 7. Model 6 — Emotion FERPlus ONNX (Facial Emotion CNN)
- 8. Model 7 — OpenCV Haar Cascades (Face/Eye/Smile Detection)
- 9. LLM Provider Routing & Fallback Architecture
- 10. Backend Libraries & Frameworks
- 11. Frontend Libraries & Frameworks
- 12. Database (MongoDB)
- 13. Environment Configuration (.env)
- 14. System Architecture Diagram
- 15. File-to-Model Quick Reference

1. AI/ML Models at a Glance

The InterviewAI project uses **7 AI/ML models** across 4 different domains: Language Understanding (LLM), Speech Recognition (ASR), Semantic Analysis (Embeddings), and Computer Vision (Emotion & Eye Contact).

#	Model	Task	Runs On
1	Qwen/Qwen3-8B	Interview questions, answer evaluation, counter-questioning	HuggingFace Cloud API
2	Qwen/Qwen3-4B	Fallback LLM (when 8B model fails)	HuggingFace Cloud API
3	Ollama Qwen2.5:7B	Local LLM (works without internet)	Local Machine (Ollama)
4	openai/whisper-large-v3-turbo	Voice → Text (Speech Recognition)	HuggingFace Cloud API
5	BAAI/bge-small-en-v1.5	Answer relevance check (Semantic Similarity)	HuggingFace Cloud API
6	emotion-ferplus-8.onnx	Facial Emotion Recognition (CNN)	Local (ONNX via OpenCV)
7	OpenCV Haar Cascades	Face, Eye, Smile Detection	Local (OpenCV)

2. Model 1 — Qwen/Qwen3-8B (Primary Large Language Model)

This is the **primary LLM** — it handles all intelligent interview operations including question generation, answer evaluation, counter-questioning, resume analysis, and feedback generation.

Property	Value
Model ID	Qwen/Qwen3-8B
Provider	HuggingFace Router API (Cloud)
Parameters	8 Billion
Type	Causal Language Model (Chat Completions)
API Endpoint	https://router.huggingface.co/v1/chat/completions
Config Variable	HF_LLM_MODEL (in .env)
Source File	backend/app/ai/hf_client.py
Timeout	120 seconds
Temperature	0.3 (default)
Max Tokens	1024 (default)

What This Model Does

- **Resume-based Opening Question:** Reads candidate's resume and generates the first project-based question
- **Counter-Questions:** Analyzes candidate's answer and generates relevant follow-up questions
- **Answer Evaluation:** Scores each answer (0-100) with strengths/weaknesses analysis
- **ATS Resume Analysis:** Extracts skills, projects, and structured context from resume text
- **Final Feedback:** Generates STAR-method recommendations after interview completion

Qwen3 Special Handling

■■ Qwen3 models include a <think>...</think> block by default ("thinking mode"). We append */no_think* to all prompts to get direct answers. The *parse_json_from_llm_response()* function in *llm_service.py* automatically strips these tags from responses.

```
if "qwen" in model.lower():
    user_content = prompt + " /no_think"
```

3. Model 2 — Qwen/Qwen3-4B (Fallback LLM)

This is the **automatic fallback LLM** — when the primary Qwen3-8B model fails (timeout, server down, rate limit), the system automatically retries with this smaller model.

Property	Value
Model ID	Qwen/Qwen3-4B
Parameters	4 Billion
Config Variable	HF_LLM_FALLBACK_MODEL (in .env)
Source File	backend/app/ai/llm_service.py (Line 70-95)
Same API Endpoint	https://router.huggingface.co/v1/chat/completions

Fallback Flow

Step 1: Call Qwen3-8B (primary) → SUCCESS? Return response
Step 2: Qwen3-8B FAILED → Call Qwen3-4B (fallback) → SUCCESS? Return response
Step 3: Both FAILED → Return None (callers use mock/graceful degradation)

4. Model 3 — Ollama Qwen2.5:7B (Local/Offline LLM)

This is the **local/offline LLM** — works without internet. Requires Ollama runtime installed on the developer's machine. Useful for development, testing, and environments without internet.

Property	Value
Model ID	qwen2.5:7b
Provider	Ollama (Local Runtime)
Parameters	7 Billion
API Endpoint	http://localhost:11434/api/chat
Config Variables	OLLAMA_MODEL, OLLAMA_BASE_URL
Source File	backend/app/ai/ollama_client.py
Timeout	120 seconds

How to Activate Ollama

```
# Step 1: Install Ollama (https://ollama.ai)
# Step 2: Pull the model
ollama pull qwen2.5:7b

# Step 3: Start Ollama server
ollama serve

# Step 4: Set in .env
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:7b
```

5. Model 4 — OpenAI Whisper-large-v3-turbo (Speech-to-Text)

This is the **Automatic Speech Recognition (ASR)** model — converts candidate's spoken voice into text transcription for evaluation.

Property	Value
Model ID	openai/whisper-large-v3-turbo
Provider	HuggingFace Inference API
Type	Automatic Speech Recognition (ASR)
API Endpoint	https://router.huggingface.co/hf-inference/models/openai/whisper-large-v3-turbo
Config Variable	HF_WHISPER_MODEL
Source File	backend/app/ai/speech_service.py
Input Format	16kHz, 16-bit, Mono WAV
Timeout	180 seconds

Audio Pipeline Flow

```
Browser Microphone (MediaDevices API)
↓
Web Audio API (Float32 PCM samples)
↓
In-Browser WAV Encoder (16kHz, 16-bit, Mono) [InterviewRoom.jsx]
↓
WebSocket → FastAPI Backend
↓
HuggingFace Whisper API → Transcript Text
```

Fallback Chain

If Whisper API fails or HF_TOKEN is missing:

1. **Browser Transcript** — Uses Web Speech API transcript from the browser
2. **Smart Mock** — Generates contextually relevant mock answer based on question keywords

■■ Audio Format: Browsers record WebM/Opus by default, but Whisper requires WAV. We encode 16kHz WAV in-browser using Web Audio API (in InterviewRoom.jsx) before sending to the server.

6. Model 5 — BAAI/bge-small-en-v1.5 (Semantic Embeddings)

This **Semantic Similarity** model measures how relevant a candidate's answer is to the question asked, by comparing their vector representations.

Property	Value
Model ID	BAAI/bge-small-en-v1.5
Provider	HuggingFace Inference API (Feature Extraction)
Type	Sentence Embedding Model
Output Dimensions	384-dimensional vector per text
API Endpoint	https://router.huggingface.co/hf-inference/models/BAAI/bge-small-en-v1.5

Property	Value
Config Variable	HF_EMBEDDING_MODEL
Source File	backend/app/ai/embedding_service.py
Timeout	30 seconds

How It Works



Fallback: When embedding API is unavailable, Jaccard word-overlap similarity is used (score range: 30-85).

7. Model 6 — Emotion FERPlus ONNX (Facial Emotion Recognition CNN)

This **deep Convolutional Neural Network (CNN)** detects facial emotions from webcam frames in real-time. It runs locally using OpenCV's DNN module.

Property	Value
Model File	emotion-ferplus-8.onnx (35 MB)
Location	backend/app/models/emotion-ferplus-8.onnx
Type	Convolutional Neural Network (ONNX format)
Input	64×64 grayscale face image
Output	8 emotion class probabilities
Runtime	OpenCV DNN module (cv2.dnn.readNetFromONNX)
Source File	backend/app/services/cv_service.py
Runs	Locally — no API call needed

Detected Emotions (FERPlus Classes)

Index	FERPlus Class	Our Mapping
0	Neutral	neutral
1	Happiness	happy
2	Surprise	surprise
3	Sadness	sad
4	Anger	angry
5	Disgust	disgust (combined with contempt)
6	Fear	fear
7	Contempt	merged into disgust

Processing Pipeline

1. Detect face using Haar Cascade → crop face ROI
 2. Convert to grayscale
 3. Resize to 64×64 pixels
 4. Reshape to blob: (1, 1, 64, 64)
 5. Run ONNX CNN inference: net.setInput(blob) → net.forward()
 6. Apply Softmax to logits → emotion probabilities
 7. Reinforce with Haar Smile detector (if smile detected → boost "happy")

8. Model 7 — OpenCV Haar Cascades (Face/Eye/Smile Detection)

These are **pre-trained Haar feature-based cascade classifiers** built into OpenCV. They detect face, eyes, and smile using gradient features on grayscale images.

Cascade Classifier	OpenCV File	Purpose
Frontal Face	haarcascade_frontalface_default.xml	Detects face from front view

Cascade Classifier	OpenCV File	Purpose
Profile Face	haarcascade_profileface.xml	Detects face from side angle
Eye	haarcascade_eye.xml	Detects eyes (for eye contact tracking)
Smile	haarcascade_smile.xml	Detects smile (boosts 'happy' emotion)

Eye Contact Score Algorithm

```
Base score = 86.0
if num_eyes >= 2: score += 10.0 # Both eyes visible
if num_eyes == 1: score -= 6.0 # Slight angle
if num_eyes == 0: score -= 32.0 # Looking away

# Centering penalty (face should be centered in camera frame)
centering_penalty = max(0, (max(dx, dy) - 0.35)) × 40.0
score -= centering_penalty
Final range: 15.0 to 98.0
```

9. LLM Provider Routing & Fallback Architecture

All LLM calls go through a single central function `call_llm()` in `llm_service.py`. It handles provider selection and automatic fallback.

```
call_llm(prompt)
■
■■■ AI MOCK_MODE = true?
■   ■■■ YES → Skip all API calls, return None
■
■■■ LLM_PROVIDER = "ollama"?
■   ■■■ YES → Call Ollama Qwen2.5:7b (local)
■       ■   ■■■ FAILED? → Fallback to HuggingFace chain
■       ■
■       ■■■ NO → HuggingFace (default)
■           ■■■ Primary: Qwen3-8B
■           ■   ■■■ FAILED? → Fallback: Qwen3-4B
■           ■   ■■■ FAILED? → return None
■           ■■■ SUCCESS → return response
```

Provider Switching

Environment Variable	Value	Effect
LLM_PROVIDER	huggingface	Uses HuggingFace Cloud API (Qwen3-8B → Qwen3-4B fallback)
LLM_PROVIDER	ollama	Uses local Ollama runtime (no internet needed)
AI MOCK_MODE	true	Skips all real AI calls; callers use mock/fallback data
AI MOCK_MODE	false	Normal operation — real AI models are called

10. Backend Libraries & Frameworks

Core Framework

Library	Version	Purpose
FastAPI	0.115.8	Async web framework (REST + WebSocket)
Uvicorn	0.34.0	ASGI server (runs FastAPI)
Pydantic	2.10.6	Data validation & serialization
pydantic-settings	2.7.1	Loads config from .env file
Motor	3.6.0	Async MongoDB driver
PyMongo	4.10.1	MongoDB Python driver (Motor depends on it)
websockets	≥12.0	WebSocket support for FastAPI

Authentication & Security

Library	Version	Purpose
bcrypt	4.2.1	Password hashing
passlib	1.7.4	Password hashing utilities
python-jose	3.3.0	JWT token creation & verification

AI & Machine Learning

Library	Version	Purpose
httpx	≥0.27.0	Async HTTP client (all HuggingFace API calls)
openai	1.61.1	OpenAI-compatible client (legacy, not actively used)
opencv-python-headless	4.11.0.86	Computer vision (face/eye/emotion + ONNX runtime)
numpy	2.2.2	Numerical computation (audio energy, softmax, embeddings)
soundfile	0.13.1	WAV audio file reading & voice analysis
librosa	0.10.2	Advanced audio analysis (optional, fallback)
deepface	0.0.93	Face analysis library (optional, heavy dependency)
tf-keras	2.18.0	TensorFlow/Keras backend for DeepFace (optional)

Document Processing

Library	Version	Purpose
pypdf	6.16.2	PDF resume text extraction
python-multipart	0.0.20	File upload handling
reportlab	4.3.1	PDF report generation (score cards, radar charts)

Utilities

Library	Version	Purpose
python-dotenv	1.0.1	.env file loading
gdown	5.2.0	Google Drive file download (model downloads)

11. Frontend Libraries & Frameworks

NPM Dependencies

Library	Version	Purpose
React	19.2.8	UI component framework
React DOM	19.2.8	DOM rendering
React Router DOM	7.18.2	Client-side routing (SPA navigation)
Vite	8.2.0	Build tool & dev server
Tailwind CSS	4.3.3	Utility-first CSS framework
Axios	1.19.0	HTTP client (REST API calls to backend)
Framer Motion	13.1.1	Smooth animations & transitions
Lucide React	1.33.0	Icon library
Recharts	3.10.1	Data visualization (radar charts, score graphs)
PostCSS	8.5.26	CSS processing
Autoprefixer	10.5.4	CSS vendor prefix automation
OxLint	1.75.0	Fast JavaScript linter

Browser APIs (No Library Needed)

API	Purpose
Web Audio API	Microphone audio capture & 16kHz WAV encoding in browser
Web Speech API	Browser-native speech recognition (fallback for Whisper)
HTML5 Canvas	Webcam video capture & frame extraction for emotion analysis
WebSocket API	Real-time full-duplex interview communication
MediaDevices API	Camera & microphone hardware access

12. Database (MongoDB)

Property	Value
Database	MongoDB Community Server
Driver	Motor 3.6.0 (Async Python)
Connection URL	mongodb://localhost:27017
Database Name	interview_ai
Config File	backend/app/database.py

Collections

Collection	Purpose	Key Fields
users	User accounts	email, first_name, last_name, hashed_password, created_at
resumes	Uploaded resumes	parsed_text, skills, resume_context, ats_score, ats_matched_keywords
interviews	Interview sessions	questions, scores_breakdown, feedback, status, overall_score, duration_seconds

13. Environment Configuration (.env)

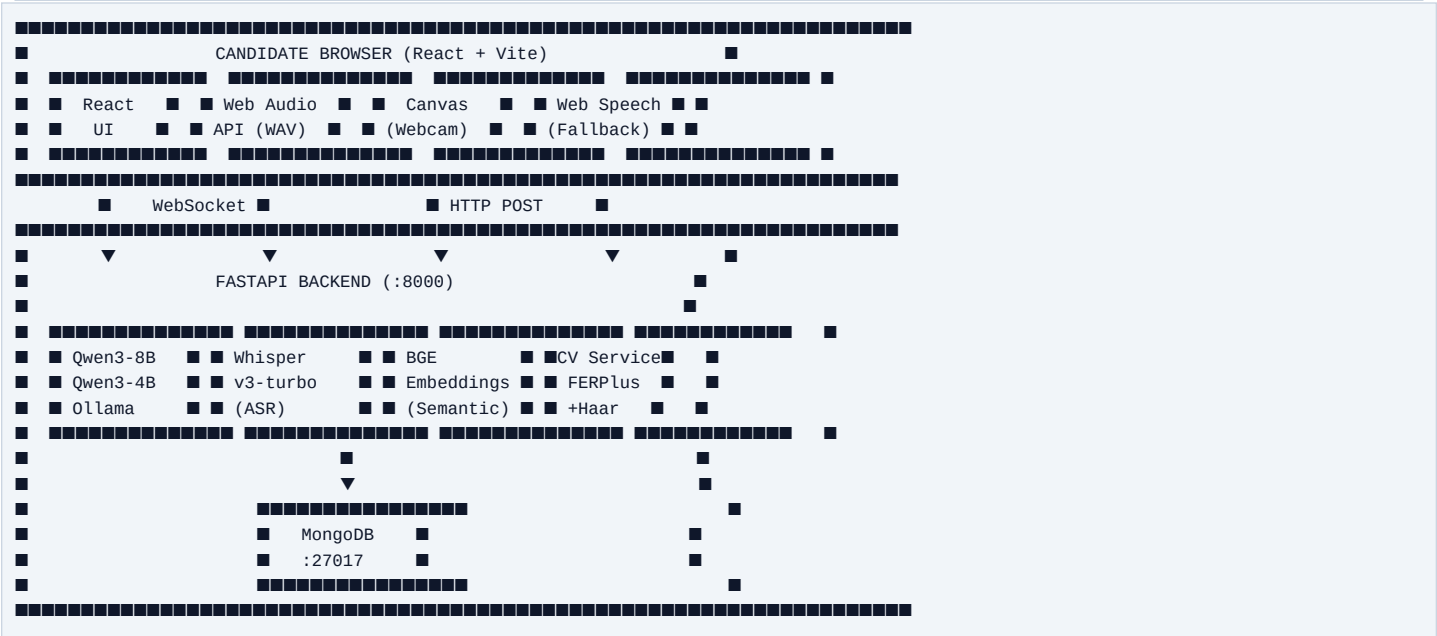
■ ■ The `.env` file must NEVER be committed to Git! It contains secrets like `HF_TOKEN` and `SECRET_KEY`.

Variable	Default Value	Description
HOST	0.0.0.0	Server bind host
PORT	8000	Server port
SECRET_KEY	(set in .env)	JWT signing secret key
ACCESS_TOKEN_EXPIRE_MINUTES	120	JWT token expiry time
MONGODB_URL	mongodb://localhost:27017	MongoDB connection string
DATABASE_NAME	interview_ai	MongoDB database name
LLM_PROVIDER	huggingface	"huggingface" or "ollama"
OLLAMA_BASE_URL	http://localhost:11434	Ollama server URL
OLLAMA_MODEL	qwen2.5:7b	Ollama model name
HF_TOKEN	(set in .env)	HuggingFace API token (get from huggingface.co/settings/tokens)
HF_LLM_MODEL	Qwen/Qwen3-8B	Primary LLM model ID
HF_LLM_FALLBACK_MODEL	Qwen/Qwen3-4B	Fallback LLM model ID
HF_WHISPER_MODEL	openai/whisper-large-v3-turbo	Speech-to-Text model ID
HF_EMBEDDING_MODEL	BAAI/bge-small-en-v1.5	Semantic embedding model ID
AI MOCK_MODE	false	Skip all AI calls and use mock data
ATS_ENABLED	true	Toggle ATS resume scoring
MAX_INTERVIEW_QUESTIONS	10	Max questions per interview session

How to Get HuggingFace Token

1. Go to <https://huggingface.co/settings/tokens>
2. Create a new token with "Read" access
3. Copy the token (starts with `hf_`)
4. Paste in `backend/.env` as `HF_TOKEN=hf_your_token_here`

14. System Architecture — How All Models Connect



15. File-to-Model Quick Reference

Source File	What It Contains
backend/app/config.py	All model names, API URLs, and environment settings
backend/app/ai/hf_client.py	HuggingFace API client (LLM, Whisper, Embeddings)
backend/app/ai/ollama_client.py	Local Ollama LLM client
backend/app/ai/llm_service.py	LLM provider routing & fallback logic (call_llm)
backend/app/ai/speech_service.py	Whisper ASR transcription + fallback chain
backend/app/ai/embedding_service.py	BGE semantic similarity scoring
backend/app/services/cv_service.py	FERPlus ONNX CNN + Haar Cascades (emotion + eye contact)
backend/app/services/audio_service.py	Voice metrics (WPM, pauses, fillers) via Soundfile + NumPy
backend/app/ai/interview_engine.py	Core interview logic (question generation, counter-questioning)
backend/app/ai/resume_parser.py	Resume text extraction & structured context
backend/app/ai/ats_scorer.py	ATS keyword scoring engine
backend/app/ai/prompt_templates.py	All LLM prompt templates
frontend/src/pages/InterviewRoom.jsx	In-browser 16kHz WAV encoding + WebSocket + Camera
frontend/src/components/CameraPanel.jsx	Webcam capture & frame extraction for emotion analysis

■ **For New Teammates:** Start with README.md for setup instructions, then read this document for model details. Follow the source file paths above to dive deeper into any specific model's implementation.